

★ Machine Learning Algorithms

● Min-Max and Z-score Normalization

↓ use the two methods below to normalize the following group of data: 1000, 2000, 3000, 5000, 9000

- ↓ Min-Max Normalization by setting $\text{Min} = 0$ and $\text{Max} = 1$
- ↓ Z-score Normalization

Data (V) : 1000 | 2000 | 3000 | 5000 | 9000 with newmax and newmin are 1 & 0.

if we have

$\text{min} = 1000$ and $\text{max} = 9000$.

we have min-max normalization

$$V = \frac{x - \text{min}}{\text{max} - \text{min}} (\text{newmax} - \text{newmin}) + \text{newmin}$$

for, $x = 1000$,

$$V = \frac{1000 - 1000}{9000 - 1000} (1 - 0) + 0$$

$$V = 0$$

for, $x = 2000$,

$$V = \frac{2000 - 1000}{9000 - 1000} (1 - 0) + 0$$

$$V = 0.125$$

for, $x = 3000$,

$$V = \frac{3000 - 1000}{9000 - 1000} (1 - 0) + 0$$

$$V = 0.25$$

for, $x = 5000$

$$V = \frac{5000 - 1000}{9000 - 1000} (1 - 0) + 0$$

$$V = 0.5$$

for $x = 9000$

$$V = \frac{9000 - 1000}{9000 - 1000} (1 - 0) + 0$$

$$V = 1$$

Original data

$$\text{Data}(V) = [1000 | 2000 | 3000 | 5000 | 9000]$$

Normalized data

$$\text{data}(V) = [0 | 0.125 | 0.25 | 0.5 | 1]$$

• Z-score Normalization

$$Z = \frac{x - \mu}{\sigma} \quad \text{whereas,}$$

$\mu = \text{mean}$

$\sigma = \text{Standard deviation}$

$$\text{Mean} = \frac{x_1 + x_2 + x_3 + x_4 + \dots + x_n}{n}$$

$$\text{Mean} = \frac{1000 + 2000 + 3000 + 5000 + 9000}{5} = 4000$$

$$\text{Standard deviation} = \sqrt{\frac{\sum (x_i - \mu)^2}{n - 1}}$$

$$= \sqrt{\frac{(1000 - 4000)^2 + (2000 - 4000)^2 + (3000 - 4000)^2 + (5000 - 4000)^2 + (9000 - 4000)^2}{5 - 1}}$$

$$= 2489.97$$

for, $x = 1000$

$$Z = \frac{1000 - 4000}{2489.97} = -1.204$$

$$\text{for } V = 2000$$

$$V = \frac{2000 - 4000}{2489.97} = -0.803$$

$$\text{for } V = 3000$$

$$V = \frac{3000 - 4000}{2489.97} = -0.4016$$

$$\text{for } V = 5000$$

$$V = \frac{5000 - 4000}{2489.97} = 0.4016$$

$$\text{for } V = 9000$$

$$V = \frac{9000 - 4000}{2489.97} = 2.008$$

So, we have now,

Original data

Data(V):

1000	2000	3000	5000	9000
------	------	------	------	------

Normalized data

Data(V):

-1.204	-0.803	-0.4016	0.4016	2.008
--------	--------	---------	--------	-------

Coding (Min-Max Normalization)

from sklearn.preprocessing import MinMaxScaler

import numpy as np

import pandas as pd

data = pd.DataFrame({'Age': [1000, 2000, 3000, 5000, 9000]})

scaler = MinMaxScaler()

data['Age-MinMax'] = scaler.fit_transform(data[['Age']])

print(data)

Coding (Z-score Normalization)

from sklearn.preprocessing import StandardScaler
import numpy as np

import pandas as pd

data = pd.DataFrame({

 'Age': [1000, 2000, 3000, 5000, 9000]

})

scaler = StandardScaler()

data['Age_zscore'] = scaler.fit_transform(data[['Age']])

print(data)

• Gaussian Elimination Method Algorithm

→ Gaussian elimination method is a technique used to solve a system of linear equations by converting the equations into an upper triangular matrix and then finding the unknown values using back substitution.

→ Where we used in ML?

1. Linear Regression for solving normal Equations.
2. Least square problems for minimizing Prediction Error
3. Matrix operation
4. Optimization Algorithms

Solve the Problems using Gaussian elimination method:

$$2x_1 + 4x_2 = 6$$

$$4x_1 + 3x_2 = 7$$

Soln $\rightarrow$

$$A = \begin{bmatrix} 2 & 4 \\ 4 & 3 \end{bmatrix} \begin{bmatrix} 6 \\ 7 \end{bmatrix}$$

$$R_1 \rightarrow R_1 / 2$$

$$A = \begin{bmatrix} 1 & 2 \\ 4 & 3 \end{bmatrix} \begin{bmatrix} 3 \\ 7 \end{bmatrix}$$

$$R_2 \rightarrow R_2 - 4R_1$$

$$A = \begin{bmatrix} 1 & 2 \\ 0 & -5 \end{bmatrix} \begin{bmatrix} 3 \\ -5 \end{bmatrix}$$

$$R_2 \rightarrow R_2 / -5$$

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

$$R_1 \rightarrow R_1 - 2R_2$$

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

now, you can say

$$x_1 = 1$$

$$x_2 = 1$$

Codeing

import numpy as np

A = np.array([
[2, 4],
[4, 3]

Coefficient Matrix

1)

B = np.array([6, 7])

Constant Matrix

X = np.linalg.solve(A, B)

Solve $AX = B$

print("X1 = ", X[0])

X1 = 1.0

print("X2 = ", X[1])

X2 = 1.0

● LU Decomposition Matrix

- > One of the simplest matrix decomposition is LU decomposition where the matrix A can be decomposed matrix.

$A = LU$ whereas $L \rightarrow$ lower triangular matrix

$U \rightarrow$ upper triangular matrix

- > The decomposition can be done using gaussian elimination methods

Step 1:

An identity matrix is augmented to the given matrix.

Step 2: Now operation and Gaussian elimination method is applied to reduce the given matrix to get matrices L and U .

Solve the problems:

$$A = \begin{bmatrix} 1 & 2 & 4 \\ 3 & 3 & 2 \\ 3 & 4 & 2 \end{bmatrix}$$

Step 1: Identity matrix

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 3 & 3 & 2 \\ 3 & 4 & 2 \end{bmatrix}$$

$$R_2 = R_2 - 3R_1$$

$$A = \begin{bmatrix} 1 & 0 & 0 \\ -3 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 3 & 4 & 2 \end{bmatrix}$$

$$R_3 = R_3 - 3R_1$$

$$A = \begin{bmatrix} 1 & 0 & 0 \\ -3 & 1 & 0 \\ -3 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & -2 & -10 \end{bmatrix}$$

$$R_3 = R_3 + R_1$$

$$R_3 = -3R_3 + 2R_2$$

$$A = \begin{bmatrix} 1 & 0 & 0 \\ -3 & 1 & 0 \\ 3 & 2 & -3 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & 0 & 10 \end{bmatrix}$$

$$R_1 = 3R_1 + 2R_2$$

$$\begin{bmatrix} -3 & 2 & 0 \\ -3 & 1 & 0 \\ 3 & 2 & -3 \end{bmatrix} \begin{bmatrix} 3 & 0 & -8 \\ 0 & -3 & -10 \\ 0 & 0 & 10 \end{bmatrix}$$

$$R_2 = R_2 + R_3$$

$$\begin{bmatrix} -3 & 2 & 0 \\ 0 & 3 & -3 \\ 3 & 2 & -3 \end{bmatrix} \begin{bmatrix} 3 & 0 & -8 \\ 0 & -3 & 0 \\ 0 & 0 & 10 \end{bmatrix}$$

$$R_1 = 10R_1 + 8R_3$$

$$\begin{bmatrix} -6 & 36 & -24 \\ 0 & 3 & -3 \\ 3 & 2 & -3 \end{bmatrix} \begin{bmatrix} 30 & 0 & 0 \\ 0 & -3 & 0 \\ 0 & 0 & 10 \end{bmatrix}$$

$$R_1 = R_1 / 3, \quad R_2 = -R_2 / 3$$

$$\begin{bmatrix} -2 & 12 & -8 \\ 0 & -1 & 1 \\ 3 & 2 & -3 \end{bmatrix} \begin{bmatrix} 10 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 10 \end{bmatrix}$$

$$R_1 = R_1 / 2$$

$$\begin{bmatrix} -1 & 6 & -4 \\ 0 & -1 & 1 \\ 3 & 2 & -3 \end{bmatrix} \begin{bmatrix} 5 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 10 \end{bmatrix}$$

● Principal Component Analysis (PCA)

> Principal Component Analysis (PCA) is a dimensionality reduction technique used to reduce the number of features while preserving as much information (variance) as possible.

> Instead of using the original features, PCA creates new features called principal components.

> Goals:

1. Reduce dimensions.
2. Remove redundancy
3. Reduce computation time
4. Visualize high-dimensional data

> PCA used in Machine Learning

1. Data Compression

> Reduce 100 features to 10-20 features while retaining most information.

2. Image Processing

> face recognition, image compression.

3. Visualization

> Convert high-dimensional data to 2D or 3D for plotting

4. Noise Reduction

> Remove less important information.

5. Before Training Models

- > K-Means Algorithm Improved
- > Logistic Regression Algorithm Improved
- > Support Vector Machine Improved
- > Neural Networks Improved

Solve the Problems:

feature	Example 1	Example 2	Example 3	Example 4
X_1	4	8	13	7
X_2	11	4	5	14

Step 1: Calculate Mean of both features (X_1, X_2).

$$X_1 = \frac{4 + 8 + 13 + 7}{4} = 8$$

$$X_2 = \frac{11 + 4 + 5 + 14}{4} = 8.5$$

Step 2: Calculate the covariance matrix

$$S = \begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) \end{bmatrix}$$

$$> \text{Cov}(X_1, X_1) = \frac{1}{N-1} \sum_{k=1}^N (X_{1k} - X_1)(X_{1k} - X_1)$$

$$\Rightarrow \frac{1}{3} \cdot ((4-8)^2 + (8-8)^2 + (13-8)^2 + (7-8)^2)$$

$$\Rightarrow 14$$

$$> \text{Cov}(X_1, X_2) = \frac{1}{N-1} \sum_{k=1}^N (X_{1k} - X_1)(X_{2k} - X_2)$$

$$\Rightarrow \frac{1}{3} ((4-8)(11-8.5) + (8-8)(4-8.5) + (13-8)(5-8.5) + (7-8)(14-8.5))$$

$$\Rightarrow -11$$

$$\therefore \text{COV}(X_2, X_1) = -11 \quad \text{same as } \text{COV}(X_1, X_2)$$

$$\therefore \text{COV}(X_2, X_2) = \frac{1}{N-1} \sum_{k=1}^N (X_{2k} - X_2)(X_{2k} - X_2)$$

$$\Rightarrow \frac{1}{3} ((11-8.5)^2 + (4-8.5)^2 + (5-8.5)^2 + (14-8.5)^2)$$

$$\Rightarrow 23$$

$$S = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

Step 3: Eigen values of the Covariance Matrix

$$0 = \det(S - \lambda I)$$

$$= \begin{vmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{vmatrix}$$

$$= (14 - \lambda)(23 - \lambda) - (-11)(-11)$$

$$= 322 - 14\lambda - 23\lambda + \lambda^2 - 121$$

$$\Rightarrow \lambda^2 - 37\lambda + 201$$

Use ~~splitting terms~~ quadratic equations

$$a = 1, b = -37, c = 201$$

$$\lambda = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$$\lambda = \frac{-(-37) \pm \sqrt{(-37)^2 - 4(1)(201)}}{2(1)}$$

$$\lambda_1 = 6.6151, \lambda_2 = 30.3849$$

Step 4: Computation of the eigenvectors

$$U = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 0 \end{bmatrix} = (S - \lambda I)U$$

$$= \begin{bmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$= \begin{bmatrix} (14 - \lambda)u_1 - 11u_2 \\ -11u_1 + (23 - \lambda)u_2 \end{bmatrix}$$

$$(14 - \lambda)u_1 - 11u_2 = 0 \quad \text{--- (1)}$$

$$-11u_1 + (23 - \lambda)u_2 = 0 \quad \text{--- (2)}$$

$$(14 - \lambda) \mu_1 - 11 \mu_2 = 0$$

$$(14 - \lambda) \mu_1 = 11 \mu_2$$

$$\therefore \frac{\mu_1}{11} = \frac{\mu_2}{14 - \lambda} = t$$

$$\therefore \mu_1 = 11t, \mu_2 = (14 - \lambda)t \quad \text{whereas } t = 1$$

$$U_1 = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \quad U_1 = \begin{bmatrix} 11 \\ 14 - \lambda \end{bmatrix}$$

Whenever we have to calculate PCA, we consider
largest eigenvalues = 30.3849.

To find unit eigenvector, we compute the length of U_1

$$\begin{aligned} \|U_1\| &= \sqrt{11^2 + (14 - \lambda)^2} \\ &= \sqrt{121 + (14 - 30.3849)^2} \\ &= 19.7348 \end{aligned}$$

Eigenvector of c_1 will be

$$e_1 = \begin{bmatrix} 11 / \|U_1\| \\ (14 - \lambda) / \|U_1\| \end{bmatrix} = \begin{bmatrix} 11 / 19.7348 \\ (14 - 30.3849) / 19.7348 \end{bmatrix}$$

$$e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$$

Step 5: Computation of first Principal Components

$$e_1^T \begin{bmatrix} X_{1k} - \bar{X}_1 \\ X_{2k} - \bar{X}_2 \end{bmatrix} = \begin{bmatrix} 0.5574 & -0.8303 \end{bmatrix} \begin{bmatrix} X_{11} - \bar{X}_1 \\ X_{21} - \bar{X}_2 \end{bmatrix}$$

$$\begin{aligned}
 &= 0.5574 (X_{11} - X_1) - 0.8303 (X_{21} - X_2) \\
 &= 0.5574 (4 - 8) - 0.8303 (11 - 8.5) \\
 &= -4.3052
 \end{aligned}$$

$$\begin{aligned}
 &\Rightarrow [0.5574 \quad -0.8303] \begin{bmatrix} X_{12} - X_1 \\ X_{22} - X_2 \end{bmatrix} \\
 &= 0.5574 (X_{12} - X_1) - 0.8303 (X_{22} - X_2) \\
 &= 0.5574 (8 - 8) - 0.8303 (4 - 8.5) \\
 &= 3.7361
 \end{aligned}$$

$$\begin{aligned}
 &\Rightarrow [0.5574 \quad -0.8303] \begin{bmatrix} X_{13} - X_1 \\ X_{23} - X_2 \end{bmatrix} \\
 &= 0.5574 (X_{13} - X_1) - 0.8303 (X_{23} - X_2) \\
 &= 0.5574 (13 - 8) - 0.8303 (5 - 8.5) \\
 &= 5.6928
 \end{aligned}$$

$$\begin{aligned}
 &\Rightarrow [0.5574 \quad -0.8303] \begin{bmatrix} X_{14} - X_1 \\ X_{24} - X_2 \end{bmatrix} \\
 &= 0.5574 (X_{14} - X_1) - 0.8303 (X_{24} - X_2) \\
 &= 0.5574 (7 - 8) - 0.8303 (14 - 8.5) \\
 &= -5.1238
 \end{aligned}$$

Feature	En 1	En 2	En 3	En 4
X_1	4	8	13	7
X_2	11	4	5	14
PCA	-4.3052	3.7361	5.6928	-5.1238

Coding

import numpy as np
from sklearn.decomposition import PCA

```
X = np.array([  
    [4, 11],  
    [8, 4],  
    [13, 5],  
    [7, 14],
```

])

```
pca = PCA(n_components=1)
```

```
X_pca = pca.fit_transform(X)
```

```
print("Principal Component values:")
```

```
print(X_pca)
```

```
print("\nExplained variance:")
```

```
print(pca.explained_variance_)
```

```
print("\nPrincipal Component vector:")
```

```
print(pca.components_)
```

● S-candidate Algorithm

1. S-Algorithm

Example	Sky	Air Temp	Humidity	Wind	Water	Forecast	E
1	Sunny	Warm	Normal	Strong	Warm	Same	Y
2	Sunny	Warm	High	Strong	Warm	Same	Y
3	Rainy	Cold	High	Strong	Warm	Change	N
4	Sunny	Warm	High	Strong	Cool	Change	Y

→ Initialize h to the most specific hypothesis in H .

$$h_0 = \langle \phi, \phi, \phi, \phi, \phi, \phi \rangle$$

Step 1:

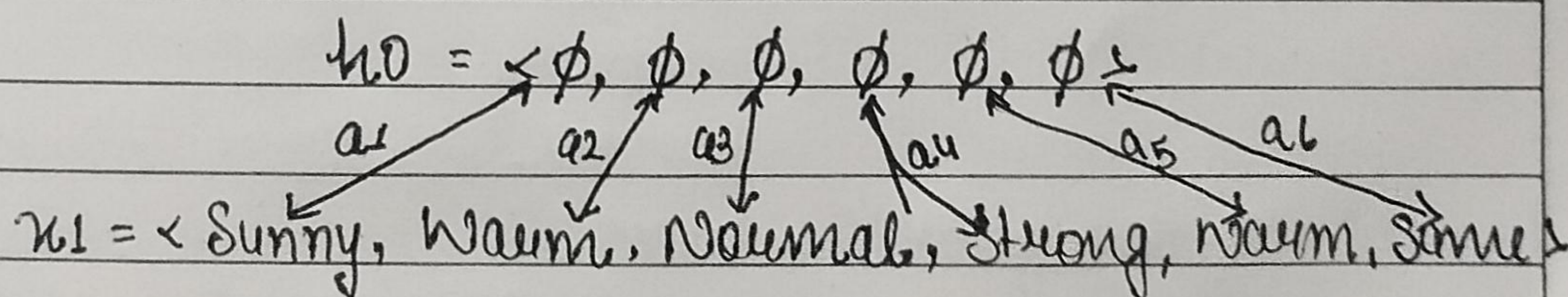
→ for each positive training instance x

• for each attribute constraint a_i in h

if constraint a_i is satisfied by x .

then do nothing

else replace a_i in h by the next more general that is satisfied by x .



Iteration 1

$h_1 = \langle \text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same} \rangle$

Step 2:

$h_1 = \langle \text{Sunny, warm, Normal, Strong, warm, same} \rangle$

Iteration 2

$x_2 = \langle \text{Sunny, warm, High, Strong, warm, same} \rangle$

$h_2 = \langle \text{Sunny, warm, ?, Strong, warm, same} \rangle$

Step 3:

Example 3 is negative training whatever hypothesis we have in Iteration 2 will be considered.

Iteration 3

Ignore

$h_3 = \langle \text{Sunny, warm, ?, Strong, warm, same} \rangle$

Step 4:

$h_3 = \langle \text{Sunny, warm, ?, Strong, warm, same} \rangle$

Iteration 4

$x_4 = \langle \text{Sunny, warm, High, Strong, cool, change} \rangle$

Output

$h_4 = \langle \text{Sunny, warm, ?, Strong, ?, ?} \rangle$

2. Candidate Algorithms

$$S_0 : \langle \phi, \phi, \phi, \phi, \phi, \phi \rangle$$

$$G_0 : \langle ?, ?, ?, ?, ?, ? \rangle$$

Steps 1: $S_1 : \langle \text{Sunny, Warm, Normal, Strong, Warm, Same} \rangle$

$$G_1 : \langle ?, ?, ?, ?, ?, ? \rangle$$

Specific to general

Steps 2:

$$S_2 : \langle \text{Sunny, Warm, ?, Strong, Warm, Same} \rangle$$

$$G_2 : \langle ?, ?, ?, ?, ?, ? \rangle$$

Example 3 is negative here and don't change anything in specific hypothesis, makes change in general hypothesis now if example value are not same just make it same like Example 2 in general hypothesis.

Step 3:

$$S_3 : \langle \text{Sunny, Warm, ?, Strong, Warm, Same} \rangle$$

$$G_3 : \{ \langle \text{Sunny, ?, ?, ?, ?, ?} \rangle, \langle ?, \text{Warm, ?, ?, ?, ?} \rangle, \langle ?, ?, ?, ?, ?, \text{Same} \rangle \}$$

Step 4: $S_4 : \langle \text{Sunny, Warm, ?, Strong, ?, ?} \rangle$

$$G_4 : \{ \langle \text{Sunny, ?, ?, ?, ?, ?} \rangle, \langle ?, \text{Warm, ?, ?, ?, ?} \rangle \}$$

Same is replaced by change in G_4
That's why we removed it.

Teacher's Signature

● Coding

import pandas as pd

```
data = [
    ['Sunny', 'warm', 'normal', 'strong', 'warm', 'same', 'yes'],
    ['Sunny', 'warm', 'high', 'strong', 'warm', 'same', 'yes'],
    ['Rainy', 'cold', 'high', 'strong', 'warm', 'change', 'no'],
    ['Sunny', 'warm', 'high', 'strong', 'cool', 'change', 'yes']
]
```

```
columns = ['Sky', 'AirTemp', 'Humidity', 'wind', 'water', 'Forecast',
            'EnjoySport']
```

```
df = pd.DataFrame(data, columns=columns)
```

```
concepts = df.iloc[:, :-1].values
```

```
target = df.iloc[:, -1].values
```

```
specific-h = concepts[0].copy()
```

```
general-h = [['?' for i in range(len(specific-h))]
              for i in range(len(specific-h))]
```

```
for i, h in enumerate(concepts):
```

```
    if target[i] == "yes":
```

```
        for u in range(len(specific-h)):
```

```
            if h[u] != specific-h[u]:
```

```
                specific-h[u] = '?'
```

```
                general-h[u][u] = '?'
```

```
    if target[i] == "no":
```

```
        for u in range(len(specific-h)):
```

```
            if h[u] != specific-h[u]:
```

```
                general-h[u][u] = specific-h[u]
```

```
    else:
```

```
        general-h[u][u] = '?'
```

```
print("Final Specific Hypothesis:")
```

```
print(specific-h)
```

```
print("Final General Hypothesis:")
```

```
for g in general-h: print(g)
```

Expt. No.:

knn Algorithm & weighted knn Algorithm

1. knn Algorithm

S.N	CGPA	Assessment	Project Result	Distance	weighted knn
				$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$	$w = 1/d$
1	9.2	85	8	P	45.20 0.0221
2	8	80	7	P	40.09 0.0248
3	8.5	81	8	P	41.18 0.0243
4	6	45	5	F	5.0 0.2000
5	6.5	50	4	F	10.05 0.0995
6	8.2	72	7	P	32.13 0.0311
7	5.8	38	5	F	2.02 0.4950
8	8.9	91	9	P	51.23 0.0195

Step 1:

Given: CGPA = 6.1

Assessment = 40

, K = 3

Project = 5

Use Euclidean distance formula:

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

find all distance values

d1 = 45.20

d7 = 2.02

d2 = 40.09

d8 = 51.23

d3 = 41.18

d4 = 5.0

d5 = 10.05

d6 = 32.13

Teacher's Signature

Step 2: Arrange in ascending order

S.N	C.G.PA	Assessment	Project	Result	distance
7	5.8	38	5	F	2.02
4	6	45	5	F	5.0
5	6.5	50	4	F	10.05
6	8.2	72	7	P	32.13
2	8	80	7	P	40.09
3	8.5	81	8	P	41.18
1	9.2	85	8	P	45.20
8	8.9	91	9	P	51.23

Step 3: Select $K=3$ nearest Neighbours

C.G.PA	Assessment	Project	Result	distance
5.8	38	5	F	2.02
6	45	5	F	5.0
6.5	50	4	F	10.05

as we can see majority of vote of $K=3$ is in favour of Fail so student is actually Fail class.

Step 4: Compute with weighted K-NN

C.G.PA	Assessment	Project	Result	distance	weighted K-NN
5.8	38	5	F	2.02	0.4950
6	45	5	F	5.0	0.2
6.5	50	4	F	10.05	0.0995

Step 5: Compute total weights

• There is no Pass result for $K=3$

• $W_{Fail} = 0.4950 + 0.2 + 0.0995 = 0.7945$

Since, no attributes of Pass student for $K=3$

Test instance belongs to Fail class.

• Coding

```
import pandas as pd
import numpy as np
from math import sqrt
```

```
data = {
    'CGPA': [9.2, 8.0, 8.5, 6.0, 6.5, 8.2, 5.8, 8.9],
    'Assessment': [85, 80, 81, 45, 50, 72, 38, 91],
    'Project': [8, 7, 8, 5, 4, 7, 5, 9],
    'Result': ['P', 'P', 'P', 'F', 'F', 'P', 'F', 'P']
}
```

```
df = pd.DataFrame(data)
```

```
test = [6.1, 40, 5]
```

```
k = 3
```

```
distances = []
```

```
for i in range(len(df)):
```

```
    d = sqrt (
```

```
        (df.iloc[i]['CGPA'] - test[0])**2 +
```

```
        (df.iloc[i]['Assessment'] - test[1])**2 +
```

```
        (df.iloc[i]['Project'] - test[2])**2
```

```
    )
```

```
    distances.append(d)
```

```
df['Distances'] = distances
```

```
df = df.sort_values(by='Distances')
```

```
print("Nearest Neighbours")
```

```
print(df.head(k))
```

```
nearest
```

```
nearest = df.head(k)
prediction = nearest['Result'].mode()[0]
print("\n KNN prediction = ", prediction)
```

```
nearest['weight'] = 1 / (nearest['Distance'] + 0.0001)
pass_weight = nearest[nearest['Result'] == 'P']['weight'].sum()
fail_weight = nearest[nearest['Result'] == 'F']['weight'].sum()
```

```
if pass_weight > fail_weight:
    weighted_prediction = 'P'
```

```
else:
    weighted_prediction = 'F'
```

```
print("\n weighted KNN Prediction = ", weighted_prediction)
```

```
print("\n Weights")
```

```
print(nearest[['Result', 'Distance', 'weight']])
```

• Nearest Centroid Classifier

Nearest Centroid Classifier

X	Y	Class
3	1	A
5	2	A
4	3	A
7	6	B
6	7	B
8	5	B

Step 1: Compute the Mean / centroid of each class

Centroid of class 'A'

$$(3+5+4, 1+2+3) / 3 = (12, 6) / 3 = (4, 2)$$

Centroid of class 'B'

$$(7+6+8, 6+7+5) / 3 = (21, 18) / 3 = (7, 6)$$

Given instance test (6, 5) to predict class

Step 2: calculate Euclidean distance formula

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

for class A

$$d = \sqrt{(4-6)^2 + (2-5)^2} = 3.6$$

for class B

$$d = \sqrt{(7-6)^2 + (6-5)^2} = 1.414$$

The test instance has smallest distance to class B.

• Coding

```
import pandas as pd
import numpy as np
from math import sqrt
```

```
data = {
    'X': [3, 5, 4, 7, 6, 8],
    'Y': [1, 2, 3, 6, 7, 5],
    'Class': ['A', 'A', 'A', 'B', 'B', 'B']
}
```

}

```
df = pd.DataFrame(data)
```

```
class_A = df[df['class'] == 'A']
```

```
class_B = df[df['class'] == 'B']
```

```
centroid_A_x = class_A['X'].mean()
```

```
centroid_A_y = class_A['Y'].mean()
```

```
centroid_B_x = class_B['X'].mean()
```

```
centroid_B_y = class_B['Y'].mean()
```

```
print("Centroid A =", (centroid_A_x, centroid_A_y))
```

```
print("Centroid B =", (centroid_B_x, centroid_B_y))
```

```
test_x = 6
```

```
test_y = 5
```

```
distance_A = sqrt(
```

```
(test_x - centroid_A_x)**2 + (test_y - centroid_A_y)**2
```

```
)
```

```
distance_B = sqrt(
```

```
(test_x - centroid_B_x)**2 + (test_y - centroid_B_y)**2
```

```
)
```

```
print("\nDistance from Centroid A =", distance-A)
print("Distance from Centroid B =", distance-B)
```

```
if distance-A < distance-B :
    prediction = 'A'
```

```
else:
```

```
    prediction = 'B'
```

```
print("\nPredicted class =", prediction)
```

● K-Means Clustering Algorithm

cluster it using K-means Algorithm with the initial values of object 2 and 5 with the coordinate values (4,6) and (12,4) as initial seeds.

Object	X	Y
1	2	4
2	4	6
3	6	8
4	10	4
5	12	4

Step 1: Initial Centroids:

C1 : (4,6)

C2 : (12,4)

Use Euclidean formula:

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Data Points	Distance Formula $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$		Cluster	New Cluster
-	(4,6)	(12,4)	-	-
(2,4)	2.83	10	C1	-
(4,6)	0	8.25	C1	-
(6,8)	2.83	7.21	C1	-
(10,4)	6.32	2	C2	-
(12,4)	8.25	0	C2	-

Step 2:

Initial Centroids:

$$c_1: (4, 6)$$

$$c_2: (12, 4)$$

New Centroids:

$$c_1 = \left(\frac{2+4+6}{3}, \frac{4+6+8}{3} \right) = (4, 6)$$

$$c_2 = \left(\frac{10+12}{2}, \frac{4+4}{2} \right) = (11, 4)$$

Use Euclidean distance formula:

Data Points	Distance formula $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$		Cluster	New Cluster
-	(4, 6)	(11, 4)	-	-
(2, 4)	2.83	9	C1	C1
(4, 6)	0	7.28	C1	C1
(6, 8)	2.83	6.40	C1	C1
(10, 4)	6.32	1	C2	C2
(12, 4)	8.25	1	C2	C2

Since, There is no changes in cluster assignment
the algorithm converges and clustering stop

Final clusters

- Cluster C1 - (2, 4), (4, 6) and (6, 8)
- Cluster C2 - (10, 4) and (12, 4)

Final centroids

- $C_1 = (4, 6)$

- $C_2 = (11, 4)$

● Coding

import pandas as pd

import numpy as np

from sklearn.cluster import KMeans

```
X = np.array([
    [2, 4],
    [4, 6],
    [6, 8],
    [10, 4],
    [12, 4]
```

```
])
```

```
initial_centroids = np.array([
    [4, 6],
    [12, 4]
```

```
])
```

```
kmeans = KMeans(
    n_clusters = 2,
    init = initial_centroids,
    n_init = 1,
    random_state = 0
```

```
)
```

```
kmeans.fit(X)
```

```
print("Cluster Labels: ")
```

```
for i in range(len(X)):
```

```
    print(f"Point {X[i]} -> Cluster {kmeans.labels_[i]}")
```

```
print("Initial centroids: ")
```

```
print(kmeans.cluster_centers_)
```